

PHP Include, Require / CRUD II

José Manuel Sánchez Álvarez

Contents

1 Update	1
1.1 Step 1: HTML Form for Car Modification	1
1.2 Step 2: PHP Script for Updating Car Details	2
1.3 Explanation	2
1.4 Security and Error Handling	3
1.5 Additional Enhancements	3

1 Update

To add the functionality to modify (update) a car's details in your CRUD application, you need both a PHP script to handle the update operation and an HTML form where users can input the new car data. Below is a step-by-step example of how you can implement this.

1.1 Step 1: HTML Form for Car Modification

First, create an HTML form (`modify_car.html`) where users can enter new values for the car's attributes. For simplicity, let's assume users know the ID of the car they want to modify.

```
<!DOCTYPE html>
<html>
<head>
  <title>Modify Car Details</title>
</head>
<body>
  <h1>Modify Car Details</h1>
  <form action="update_car.php" method="post">
    Car ID (required): <input type="text" name="car_id" required><br>
    Model: <input type="text" name="model"><br>
    Price: <input type="number" name="price" step="0.01"><br>
    <!-- Add other car attributes here -->
    <input type="submit" value="Update Car">
  </form>
</body>
</html>
```

1.2 Step 2: PHP Script for Updating Car Details

Create the PHP script (`update_car.php`) to process the form data and update the car information in the database.

```
<?php
require_once 'db_connect.php'; // Assume this file contains your PDO database connection

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $carId = $_POST['car_id'];
    $model = $_POST['model'] ?? null;
    $price = $_POST['price'] ?? null;
    // Retrieve other fields in a similar manner

    // Prepare the SQL statement
    $sql = "UPDATE Car SET model = :model, price = :price WHERE id = :carId";
    $stmt = $pdo->prepare($sql);

    // Bind parameters
    $stmt->bindParam(':carId', $carId, PDO::PARAM_INT);
    $stmt->bindParam(':model', $model);
    $stmt->bindParam(':price', $price);
    // Bind other parameters similarly

    try {
        $stmt->execute();
        echo "Car with ID $carId has been updated.";
    } catch (PDOException $e) {
        echo "Error updating car: " . $e->getMessage();
    }
}
?>
```

1.3 Explanation

- **HTML Form:** The form provides inputs for the car's ID and other attributes like model and price. The form data is sent to `update_car.php` upon submission.
- **PHP Script:**
 - The PHP script first checks if the form has been submitted.
 - It retrieves the car ID and other modified attributes from the form data.
 - A SQL `UPDATE` statement is prepared and executed to modify the car's details in the database.
 - The script uses PDO prepared statements to prevent SQL injection attacks.

1.4 Security and Error Handling

- **Validation:** Ensure that necessary validations are done, like checking if the car ID exists in the database.
- **Error Handling:** Implement error handling to manage database exceptions and form submission issues.
- **User Feedback:** Provide clear messages to the user about the success or failure of the update operation.

1.5 Additional Enhancements

- **Dynamic Data Retrieval:** For a better user experience, consider creating a mechanism to retrieve the current details of a car (e.g., through a separate form or a dropdown) and populate the form with this data for editing.
- **Comprehensive Form Handling:** Implement more comprehensive form validation on the server side, and consider adding client-side validation for immediate user feedback.

By following these steps, you can effectively add the ability to modify car details in your CRUD application, enhancing its functionality and usability.